



www.mech-lang.org

A Rust-Native Embeddable Reactive Numerical Language for Heterogeneous Computing in Robotics

Corey Montella, Aung Kant Ko, Steven McPhillimey

Motivation

- Rust’s memory safety, performance and low-level control make it a strong foundation for robotics. Numerical kernels can still become lengthy and difficult to verify when each backend requires specialized code.
- Mech is a numerical language implemented in Rust and embeddable in Rust applications. Its compiler specializes typed equations without backend-specific source annotations. A bearing-only EKF illustrates its numerical and reactive features, which also apply beyond state estimation.

Embeddable

Rust applications can embed Mech source through its Rust library, or load a compiled dylib. Both interfaces bind named inputs and retain state between updates. C++ and Python bindings are planned.

JIT compilation from Rust

```
use mech::kernel::{Backend, Kernel};
let source = include_str!("ekf.mec");
let kernel = Kernel::from_source(source)
    .input("bearing", [-0.55; 4])
    .export("state")
    .compile(Backend::Jit)?;
let mut ekf = kernel.start()?;
ekf.turn([("bearing", [-0.54; 4])]);
let state = ekf.state("state");
```

Compile once, then submit successive measurement updates. Each checked turn retains the last accepted state.

Checked dylib†

Artifact	Bytes	M/s ± MAD	Peak RSS (MiB)
Mech scalar	33,544	14.671 ± 0.008	2.69 ± 0.00
Rust scalar	50,016	21.121 ± 0.005	2.69 ± 0.00
Mech SIMD-4	33,864	34.863 ± 0.010	2.69 ± 0.00

Load an AOT library‡

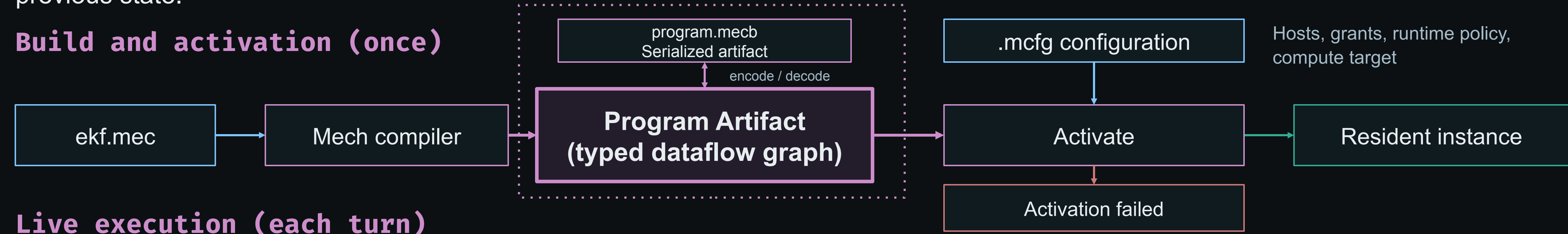
```
let kernel = unsafe { Kernel::load_bundle("ekf.bundle")? };
let mut ekf = kernel.start()?;
ekf.turn([("bearing", [-0.54; 4])]);
```

The bundle contains the dylib and its saved interface. Loading native code requires a trusted library.

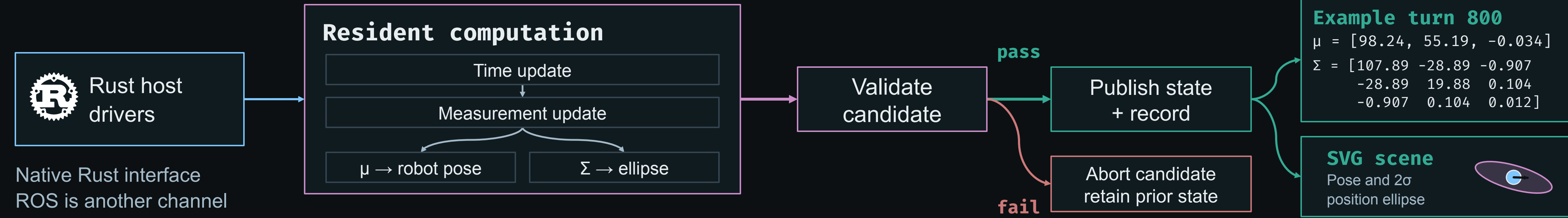
Reactive

Robots commonly use feedback control loops over continuous data streams. Mech handles successive updates through its native Rust interface or ROS channels. Each turn validates candidate state against integrity constraints. Rejection notifies the caller and retains the previous state.

Build and activation (once)



Live execution (each turn)



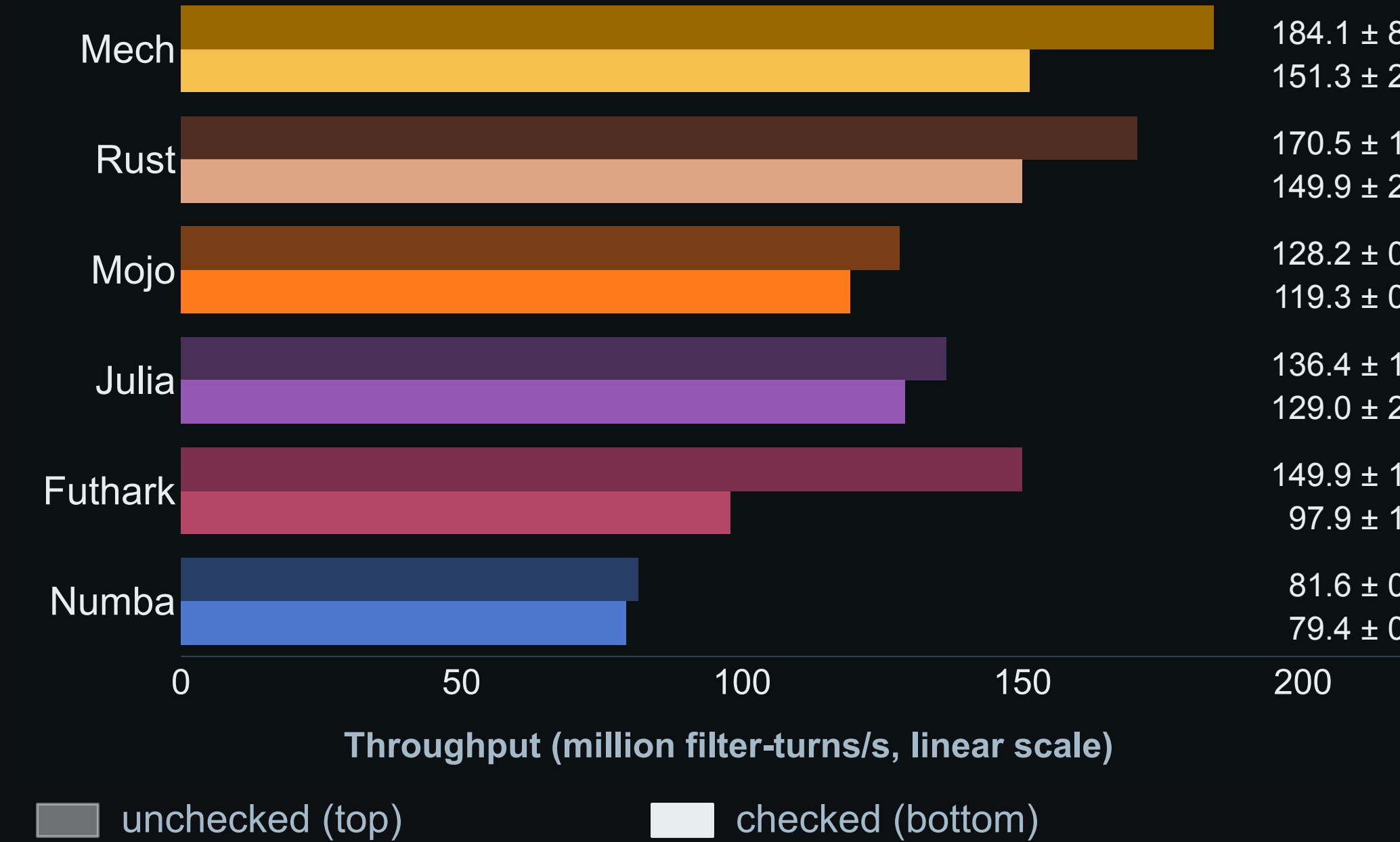
Heterogeneous

Mech’s typed, declarative IR supports interpretation, scalar/SIMD CPU, multicore, GPU and WebAssembly/WGPU from the same source.

The Mech rows use the same f32 EKF source shown at right. JIT and AOT share the same numerical compilation pipeline.‡

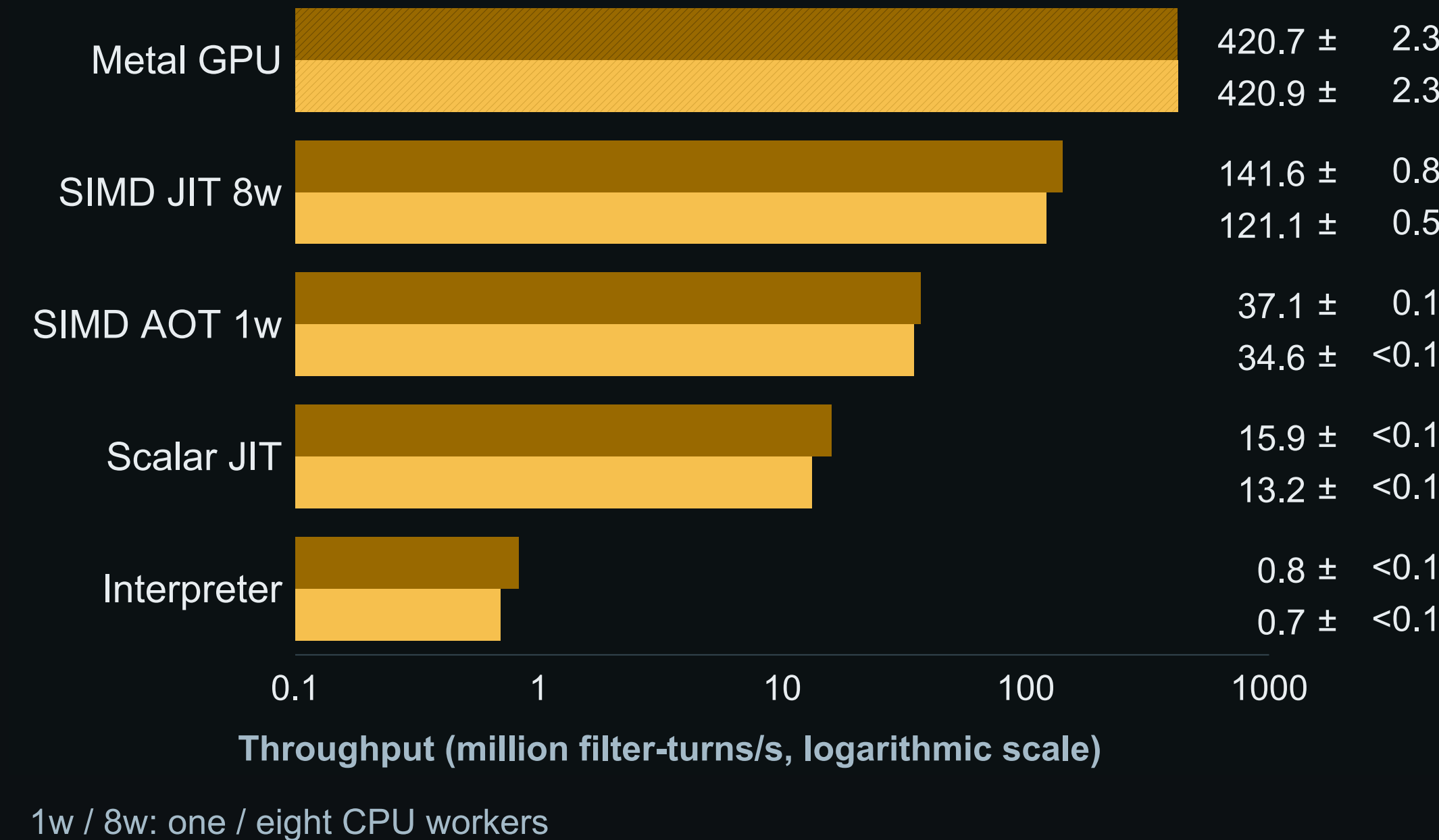
Mech and Rust show similar throughput

8 workers, fused execution
Median ± MAD



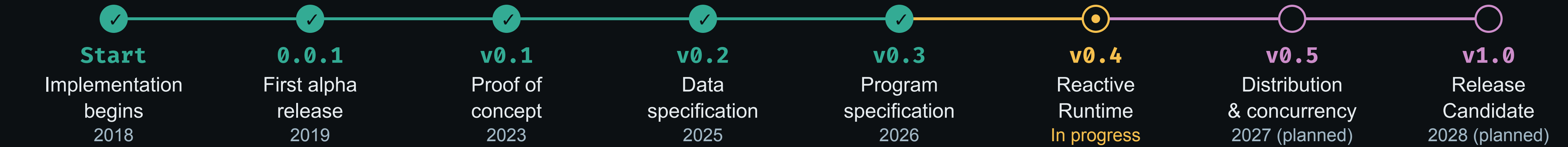
Backend choice changes EKF throughput

Per-turn publication
Median ± MAD



Status and Future Work

Mech is in beta. Further validation is needed beyond the EKF case study. Over the next year, we plan evaluation across real robot hardware platforms and in robot deployments.



Experience this poster as a live reactive program. Mech’s WebAssembly runtime supports the CPU and WGPU comparison online.



Benchmark setup. Apple M1: 4P + 4E CPU, 8-core GPU, 8 GB, macOS 15.6.1. Charts: f32, 500,000 filters × 40 turns, n=10. Median ± MAD.

Rust 1.96.0-nightly ec818fda3 / LLVM 22.1.0, Cranelift 0.131.3, Julia 1.12.6, Mojo 1.1.0 / MAX 26.6.0, Futhark 0.27.1 / ISPC 1.31.0 / LLVM 22.1.8, Python 3.12.14 / NumPy 2.2.6 / Numba 0.61.2, Apple clang 17.0.0.

Mech v0.4 development (package 0.3.6). Backend pairs f4b69052c, CPU eedc1c75b, dylibs dab98af1d, iros-workshop-benchmarks, benchmarks/iros-2026.

† Dylib table: minimal ABI loader, 1 thread, 10,000 filters × 200 turns, n=7. RSS covers the process. Compile/load/warmup excluded. File sizes reflect linker/segment layout. The new embedding wrapper is not timed.

‡ SIMD JIT and AOT share the Cranelift pipeline. JIT compiles on demand. AOT emits a library ahead of deployment, allowing the host to omit the compiler.

Unchecked runs omit integrity predicates and retain per-turn publication in the Mech chart. Checking reduces SIMD AOT throughput by 6.6%. The Metal difference is within measured variation.

[1] Thrun, Burgard & Fox. Probabilistic Robotics. MIT Press, 2005. §7.4.2, Table 7.2 (adapted).

Numerical

Mech offers a concrete syntax that expresses numerical algorithms as typed matrix equations. The bearing-only EKF below adapts the textbook formulation [1], preserving its matrix-level notation.

SIMD Rust uses almost five times as many normalized source characters here, mainly for SIMD and batch support. Reusing the Mech source across backends reduces the specialized implementations to maintain.

Compilation produces a typed dataflow artifact. Matrix operations use nalgebra in the ordinary backend; specialized backends compile the graph directly.

Extended Kalman Filter

```
+> math/*
  Δt := 0.1
  m := [140 12]'
  Q := [0.01 0
        0 0.0025]

  R := 0.25
  I := [1 0 0
        0 1 0
        0 0 1]

  ~μ := [55 25 0.4]'
  ~Σ := [100 0 0
        0 100 0
        0 0 0.15]
```

Time Update

```
θ := μ[3]
d := v * Δt
G := [1 0 (0 - d * sin(θ))
      0 1 d * cos(θ)
      0 0 1]

V := [cos(θ) * Δt 0
      sin(θ) * Δt 0
      0 Δt]
```

```
μ̄ := μ + [d * cos(θ)
          d * sin(θ)
          ω * Δt]
```

```
Σ̄ := G ** Σ ** G' + V ** Q ** V'
```

Measurement Update

```
δ := m - μ̄[1..=2]
δx := δ[1]
δy := δ[2]
q := δx ^ 2 + δy ^ 2

ẑ := atan2(δy, δx) - μ̄[3]
r := z - ẑ
v := atan2(sin(r), cos(r))

H := [δy / q, (0 - δx) / q, -1]
S := H ** Σ̄ ** H' + R
K := (Σ̄ ** H') / S[1]
A := I - K ** H

μ₊ := μ̄ + K * v
Σ₊ := A ** Σ̄ ** A' + (K ** K') * R

μ = μ₊
Σ = Σ₊
```

Normalized source characters

Application source	Mech	Rust textbook	Rust SIMD
Setup	340	486	561
EKF + candidate update	356	1,024	1,119
Validation	383	263	396
Matrix helpers	0	251	310
SIMD + batch support	0	331	2,857
Total	1,079	2,355	5,243

The time and measurement updates remain explicit matrix equations. Backend code handles vector layout and batching, so the same numerical source can serve scalar and SIMD execution.

Rust’s selected SIMD implementation includes vector adapters and a batch dispatcher with rollback. Mech delegates these operations to reusable compiler and runtime components, which explains its zero entries for helpers and SIMD support.

Programmer-chosen names count as one character per occurrence. Comments and nonliteral whitespace are removed. General library, compiler and runtime internals are excluded.

